Linear Regression with PyTorch

Part 2 of "PyTorch: Zero to GANs"

This post is the second in a series of tutorials on building deep learning models with PyTorch, an open source neural networks library developed and maintained by Facebook. Check out the full series:

1. ~~PyTorch Basics: Tensors & Gradients~~
2. ~~Linear Regression & Gradient Descent~~ (this post)
3. Coming soon.. (logistic regression, neural networks, CNNs, RNNs, GANs etc.)

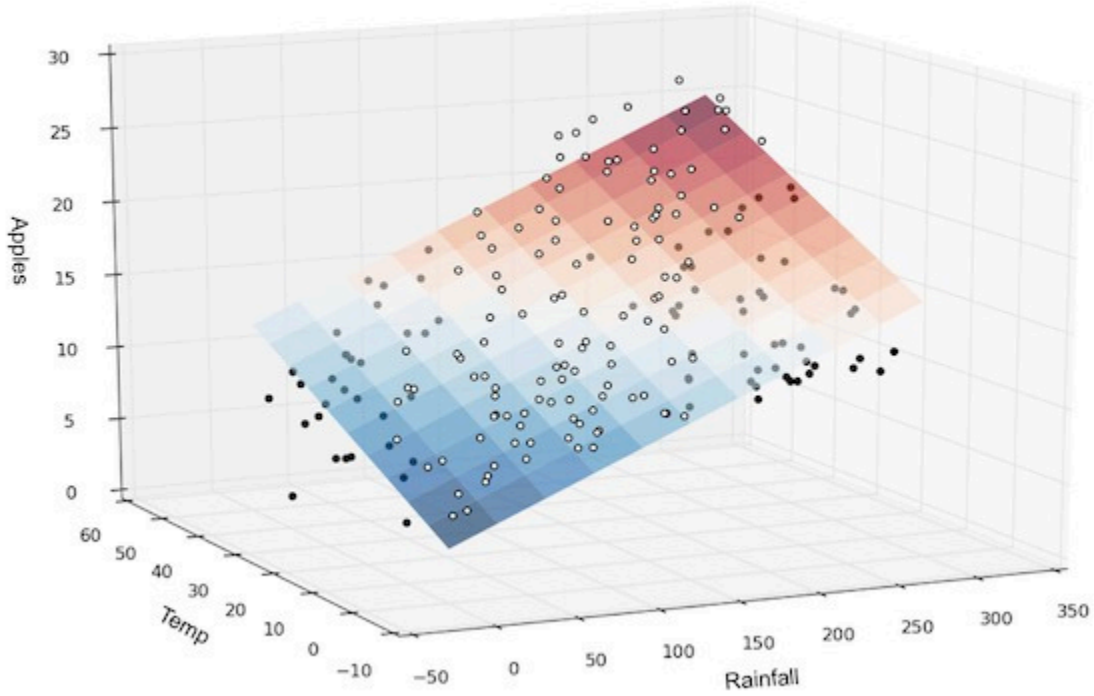
Continuing where the ~~previous tutorial~~ left off, we'll discuss one of the foundational algorithms of machine learning in this post: *Linear regression*. We'll create a model that predicts crop yields for apples and oranges (*target variables*) by looking at the average temperature, rainfall and humidity (*input variables or features*) in a region. Here's the training data:

Region	Temp. (F)	Rainfall (mm)	Humidity (%)	Apples (ton)	Oranges (ton)
Kanto	73	67	43	56	70
Johto	91	88	64	81	101
Hoenn	87	134	58	119	133
Sinnoh	102	43	37	22	37
Unova	69	96	70	103	119

In a linear regression model, each target variable is estimated to be a weighted sum of the input variables, offset by some constant, known as a bias :

```
yield_apple  = w11 * temp + w12 * rainfall + w13 * humidity + b1
yield_orange = w21 * temp + w22 * rainfall + w23 * humidity + b2
```

Visually, it means that the yield of apples is a linear or planar function of temperature, rainfall and humidity:



The *learning* part of linear regression is to figure out a set of weights `w11, w12,... w23, b1 & b2` by looking at the training data, to make accurate predictions for new data (i.e. to predict the yields for apples and oranges in a new region using the average temperature, rainfall and humidity). This is done by adjusting the weights slightly many times to make better predictions, using an optimization technique called *gradient descent*.

System setup

If you want to follow along and run the code as you read, you can clone this notebook, install the required dependencies, and start Jupyter by running the following commands on the terminal:

```
pip install jovian --upgrade   # Install the jovian library
jovian clone <notebook_id>     # Download notebook & dependencies
cd 02-linear-regression       # Enter the created directory
jovian install              # Install the dependencies
conda activate 02-linear-regression # Activate virtual environment
jupyter notebook            # Start Jupyter
```

You can find the `notebook_id` by cliking the *Clone* button at the top of this page on Jovian. On older versions of conda, you might need to run `source activate 02-linear-regression` to

activate the environment. For a more detailed explanation of the above steps, check out the *System setup* section in the previous notebook.

We begin by importing Numpy and PyTorch:

```
import numpy as np
import torch
```

Training data

The training data can be represented using 2 matrices: `inputs` and `targets`, each with one row per observation, and one column per variable.

```
#       Input       (temp,       rainfall,       humidity)
inputs = np.array([[73, 67, 43],              [91, 88, 64],
                   [87, 134, 58],            [102, 43, 37],
                  [69, 96, 70]], dtype='float32')
```

```
inputs.shape
```

```
(5, 3)
```

```
# Targets (apples, oranges)
targets = np.array([[56, 70],
                    [81, 101],
                    [119, 133],
                    [22, 37],
                    [103, 119]], dtype='float32')
targets.shape
```

```
(5, 2)
```

We've separated the input and target variables, because we'll operate on them separately. Also, we've created numpy arrays, because this is typically how you would work with training data: read some CSV files as numpy arrays, do some processing, and then convert them to PyTorch tensors as follows:

```
# Convert inputs and targets to tensors
inputs = torch.from_numpy(inputs)
targets = torch.from_numpy(targets)
print(inputs)
print(targets)
```

```
tensor([[ 73., 67., 43.],
        [ 91., 88., 64.], [ 87.,
        134., 58.], [102., 43.,
        37.], [ 69., 96., 70.]])

tensor([[ 56., 70.],
        [ 81., 101.],
        [119., 133.],
        [ 22., 37.],
        [103., 119.]])
```

Linear regression model from scratch

The weights and biases (w11, w12,... w23, b1 & b2) can also be represented as matrices, initialized as random values. The first row of w and the first element of b are used to predict the first target variable i.e. yield of apples, and similarly the second for oranges.

```
#         Weights         and         biases
w  =  torch.randn(2,  3,  requires_grad=True)
b   =   torch.randn(2,   requires_grad=True)
print(w)
print(b)
```

```
w.t().shape
```

```
tensor([[-1.1100, 0.8095, 0.2356],
        [-0.1351, 0.2889, 1.0929]], requires_grad=True)
tensor([ 0.3681, -1.1507], requires_grad=True)
torch.Size([3, 2])
```

torch.randn creates a tensor with the given shape, with elements picked randomly from a normal distribution with mean 0 and standard deviation 1.

Our *model* is simply a function that performs a matrix multiplication of the inputs and the weights w (transposed) and adds the bias b (replicated for each observation).

$$X \quad \times \quad W^T \quad + \quad b$$

$$\begin{bmatrix} 73 & 67 & 43 \\ 91 & 88 & 64 \\ \vdots & \vdots & \vdots \\ 69 & 96 & 70 \end{bmatrix} \times \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \\ b_1 & b_2 \\ \vdots & \vdots \\ b_1 & b_2 \end{bmatrix}$$

We can define the model as follows:

```
def model(x):
  return x @ w.t() + b
```

@ represents matrix multiplication in PyTorch, and the .t method returns the transpose of a tensor.

The matrix obtained by passing the input data into the model is a set of predictions for the target variables.

```
# Generate predictions
preds = model(inputs)
print(preds)
```

```
tensor([[-16.2975,   55.3399],
        [-14.3315,   81.9266],
        [ 25.9333,   89.2004],
        [-69.3304,   37.9299],
        [ 17.9788,   93.7680]], grad_fn=<AddBackward0>)
```

Let's compare the predictions of our model with the actual targets.

```
# Compare with targets
print(targets)
```

```
tensor([[ 56., 70.],
        [  81.,  101.],
        [119.,  133.],  [
        22., 37.], [103.,
        119.]])
```

You can see that there's a huge difference between the predictions of our model, and the actual values of the target variables. Obviously, this is because we've initialized our model with random

weights and biases, and we can't expect it to *just work*.

Loss function

Before we improve our model, we need a way to evaluate how well our model is performing. We can compare the model's predictions with the actual targets, using the following method:

- Calculate the difference between the two matrices (`preds and targets`).
- Square all elements of the difference matrix to remove negative values.
- Calculate the average of the elements in the resulting matrix.

The result is a single number, known as the **mean squared error** (MSE).

```
# MSE loss
def mse(t1, t2):
  diff = t1 - t2
  return torch.sum(diff * diff) / diff.numel()
```

torch.sum returns the sum of all the elements in a tensor, and the .numel method returns the number of elements in a tensor. Let's compute the mean squared error for the current predictions of our model.

```
# Compute loss
loss = mse(preds, targets)
print(loss)
```

```
tensor(4168.0908, grad_fn=<DivBackward0>)
```

Here's how we can interpret the result: *On average, each element in the prediction differs from the actualtargetbyabout117.2(squareroot of 13742)* . And that's pretty bad, considering the numbers we are trying to predict are themselves in the range 50–200. Also, the result is called the *loss*, because it indicates how bad the model is at predicting the target variables. Lower the loss, better the model.

Compute gradients

With PyTorch, we can automatically compute the gradient or derivative of the loss w.r.t. to the weights and biases, because they have `requires_grad set to True`.

```
# Compute gradients
loss.backward()
```

The gradients are stored in the .grad property of the respective tensors. Note that the derivative of the loss w.r.t. the weights matrix is itself a matrix, with the same dimensions.

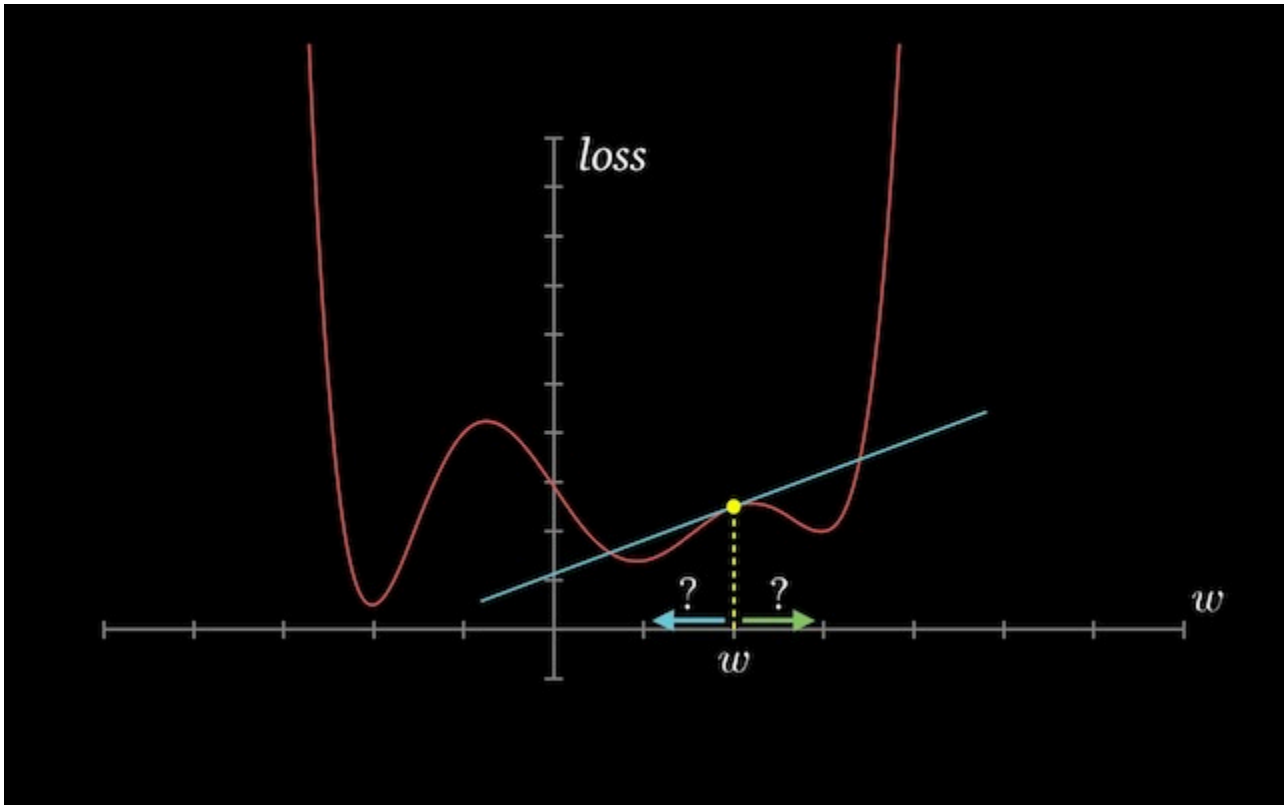
```
# Gradients for weights
print(w)
print(w.grad)
```

```
tensor([[-1.1100, 0.8095, 0.2356],
        [-0.1351, 0.2889, 1.0929]], requires_grad=True)
tensor([[-7446.3701, -7558.6577, -4787.7168],
        [-1652.5178, -2182.4233, -1224.6584]])
```

The loss is a ~~quadratic function~~ of our weights and biases, and our objective is to find the set of weights where the loss is the lowest. If we plot a graph of the loss w.r.t any individual weight or bias element, it will look like the figure shown below. A key insight from calculus is that the gradient indicates the rate of change of the loss, or the slope of the loss function w.r.t. the weights and biases.

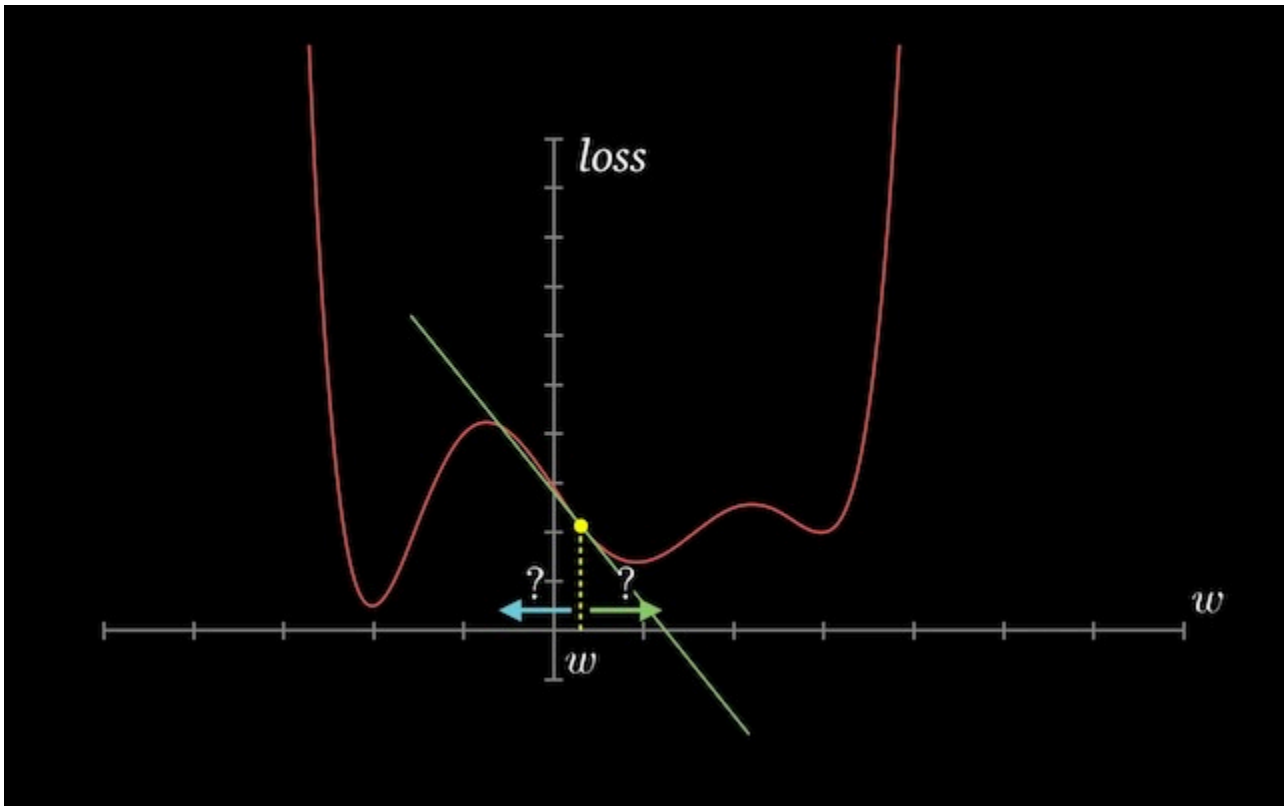
If a gradient element is **positive**:

- **increasing** the element's value slightly will **increase** the loss.
- **decreasing** the element's value slightly will **decrease** the loss



If a gradient element is **negative**:

- **increasing** the element's value slightly will **decrease** the loss.
- **decreasing** the element's value slightly will **increase** the loss.



The increase or decrease in loss by changing a weight element is proportional to the value of the gradient of the loss w.r.t. that element. This forms the basis for the optimization algorithm that we'll use to improve our model.

Before we proceed, we reset the gradients to zero by calling `.zero_()` method. We need to do this, because PyTorch accumulates, gradients i.e. the next time we call `.backward` on the loss, the new gradient values will get added to the existing gradient values, which may lead to unexpected results.

```
w.grad.zero_()
b.grad.zero_()
print(w.grad)
print(b.grad)
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([0., 0.])
```

Adjust weights and biases using gradient descent

We'll reduce the loss and improve our model using the gradient descent optimization algorithm, which has the following steps:

1. Generate predictions
2. Calculate the loss
3. Compute gradients w.r.t the weights and biases
4. Adjust the weights by subtracting a small quantity proportional to the gradient
5. Reset the gradients to zero

Let's implement the above step by step.

```
# Generate predictions
preds = model(inputs)
print(preds)
```

```
tensor([[-16.2975,   55.3399],
        [-14.3315,   81.9266],
        [ 25.9333,   89.2004],
        [-69.3304,   37.9299],
        [ 17.9788,   93.7680]], grad_fn=<AddBackward0>)
```

Note that the predictions are same as before, since we haven't made any changes to our model. The same holds true for the loss and gradients.

```
# Calculate the loss
loss = mse(preds, targets)
print(loss)
```

```
tensor(4168.0908, grad_fn=<DivBackward0>)
```

```
# Compute gradients
loss.backward()
print(w.grad)
print(b.grad)
```

```
tensor([[-7446.3701, -7558.6577, -4787.7168],
        [-1652.5178, -2182.4233, -1224.6584]])
tensor([-87.4095, -20.3670])
```

Finally, we update the weights and biases using the gradients computed above.

```
# Adjust weights & reset gradients
with torch.no_grad():
        w -
= w.grad * 1e-5
    b -= b.grad * 1e-5
```

```
    w.grad.zero_()
  b.grad.zero_()
```

A few things to note above:

- We use torch.no_grad to indicate to PyTorch that we shouldn't track, calculate or modify gradients while updating the weights and biases.
- We multiply the gradients with a really small number (10^-5 in this case), to ensure that we don't modify the weights by a really large amount, since we only want to take a small step in the downhill direction of the gradient. This number is called the *learning rate* of the algorithm.
- After we have updated the weights, we reset the gradients back to zero, to avoid affecting any future computations.

Let's take a look at the new weights and biases.

```
print(w)
print(b)
```

```
tensor([[-1.0356, 0.8851, 0.2834],
        [-0.1186, 0.3108, 1.1052]], requires_grad=True)
tensor([ 0.3690, -1.1505], requires_grad=True)
```

With the new weights and biases, the model should have lower loss.

```
# Calculate loss
preds = model(inputs)
loss = mse(preds, targets)
print(loss)
```

```
tensor(2852.2917, grad_fn=<DivBackward0>)
```

We have already achieved a significant reduction in the loss, simply by adjusting the weights and biases slightly using gradient descent.

Train for multiple epochs

To reduce the loss further, we can repeat the process of adjusting the weights and biases using the gradients multiple times. Each iteration is called an epoch. Let's train the model for 100 epochs.

```
# Train for 100 epochs
for i in range(100):
```

```
        preds = model(inputs)
    loss = mse(preds, targets)
                loss.backward()
  with torch.no_grad():      w -
= w.grad * 1e-5           b -
=      b.grad      *      1e-5
                w.grad.zero_()
    b.grad.zero_()
```

Once again, let's verify that the loss is now lower:

```
# Calculate loss
preds = model(inputs)
loss = mse(preds, targets)
print(loss)
```

```
tensor(42.5996, grad_fn=<DivBackward0>)
```

As you can see, the loss is now much lower than what we started out with. Let's look at the model's predictions and compare them with the targets.

```
# Predictions
preds
```

```
tensor([[ 55.4464, 71.6978],
        [ 79.1590, 104.0842],
        [128.3892, 122.9576],
        [ 10.5389, 46.4552],
        [102.8440, 119.3414]], grad_fn=<AddBackward0>)
```

```
# Targets
targets
```

```
tensor([[ 56., 70.],
        [  81.,  101.],
        [119.,  133.], [
        22., 37.], [103.,
        119.]])
```

The prediction are now quite close to the target variables, and we can get even better results by training for a few more epochs.

Linear regression using PyTorch built-ins

The model and training process above were implemented using basic matrix operations. But since this such a common pattern , PyTorch has several built-in functions and classes to make it easy to create and train models.

Let's begin by importing the `torch.nn` package from PyTorch, which contains utility classes for building neural networks.

```
import torch.nn as nn
```

As before, we represent the inputs and targets and matrices.

```
#          Input          (temp,          rainfall,          humidity)
inputs  =  np.array([[73,  67,  43], [91,  88,  64], [87,  134,  58],
                     [102, 43,  37], [69,  96,  70], [73,  67,  43],
                    [91,  88,  64], [87,  134,  58], [102,  43,  37],
                     [69,  96,  70], [73,  67,  43], [91,  88,  64],
                    [87,  134,  58], [102,  43,  37], [69,  96,  70]],
          dtype='float32')

#           Targets           (apples,           oranges)
targets  =  np.array([[56,  70], [81,  101], [119,  133],
                      [22,  37], [103,  119], [56,  70],
                     [81,  101], [119,  133], [22,  37],
                     [103,  119], [56,  70], [81,  101],
                    [119,  133], [22,  37], [103,  119]],
           dtype='float32')

inputs = torch.from_numpy(inputs)
targets = torch.from_numpy(targets)
```

We are using 15 training examples this time, to illustrate how to work with large datasets in small batches.

Dataset and DataLoader

We'll create a `TensorDataset`, which allows access to rows from inputs and targets as tuples, and provides standard APIs for working with many different types of datasets in PyTorch.

```
from torch.utils.data import TensorDataset


# Define dataset
train_ds = TensorDataset(inputs, targets)
train_ds[0:3]
```

```
(tensor([[ 73., 67., 43.],
         [ 91., 88., 64.],
         [ 87., 134., 58.]]),
 tensor([[ 56., 70.],
         [ 81., 101.],
         [119., 133.]]))
```

The `TensorDataset` allows us to access a small section of the training data using the array indexing notation (`[0:3]` in the above code). It returns a tuple (or pair), in which the first element contains the input variables for the selected rows, and the second contains the targets.

We'll also create a `DataLoader`, which can split the data into batches of a predefined size while training. It also provides other utilities like shuffling and random sampling of the data.

```
from torch.utils.data import DataLoader

# Define data loader
batch_size = 5
train_dl = DataLoader(train_ds, batch_size, shuffle=True)
```

The data loader is typically used in a `for-in` loop. Let's look at an example.

```
for xb, yb in train_dl:
  print(xb)  print(yb)
  break
```

```
tensor([[ 91., 88., 64.],
        [ 73., 67., 43.], [102.,
        43., 37.], [ 87., 134.,
        58.],
        [ 69., 96., 70.]])
tensor([[ 81., 101.],
        [ 56., 70.], [
        22., 37.], [119.,
        133.],
        [103., 119.]])
```

In each iteration, the data loader returns one batch of data, with the given batch size. If `shuffle` is set to `True`, it shuffles the training data before creating batches. Shuffling helps randomize the input to the optimization algorithm, which can lead to faster reduction in the loss.

nn.Linear

Instead of initializing the weights & biases manually, we can define the model using the nn.Linear class from PyTorch, which does it automatically.

```
# Define model
model = nn.Linear(3,2)
print(model.weight)
print(model.bias)
```

```
Parameter containing:
tensor([[ 0.1556, 0.0907, 0.3429],
        [ 0.4208, -0.2224, 0.5184]], requires_grad=True)
Parameter containing:
tensor([ 0.4530, -0.4931], requires_grad=True)
```

PyTorch models also have a helpful .parameters method, which returns a list containing all the weights and bias matrices present in the model. For our linear regression model, we have one weight matrix and one bias matrix.

```
# Parameters
list(model.parameters())
```

```
[Parameter containing:
tensor([[-0.0749, -0.1125, 0.0728],
        [ 0.1550, -0.0464, 0.5462]], requires_grad=True),
 Parameter containing:
 tensor([-0.0716, 0.5752], requires_grad=True)]
```

We can use the model to generate predictions in the exact same way as before:

```
# Generate predictions
preds = model(inputs)
preds
```

```
tensor([[ -9.9492,  32.2725],
        [-12.1320,  45.5597],
        [-17.4429,  39.5294],
        [ -9.8595,  34.6039],
        [-10.9465,  45.0554],
        [ -9.9492,  32.2725],
        [-12.1320,  45.5597],
        [-17.4429,  39.5294],
        [ -9.8595,  34.6039],
        [-10.9465,  45.0554],
        [ -9.9492,  32.2725],
        [-12.1320,  45.5597],
        [-17.4429,  39.5294],
        [ -9.8595,  34.6039],
        [-10.9465,  45.0554]], grad_fn=<AddmmBackward0>)
```

Loss Function

Instead of defining a loss function manually, we can use the built-in loss function `mse_loss`.

```
# Import nn.functional
import torch.nn.functional as F
```

The `nn.functional` package contains many useful loss functions and several other utilities.

```
# Define loss function
loss_fn = F.mse_loss
```

Let's compute the loss for the current predictions of our model.

```
loss = loss_fn(model(inputs), targets)
print(loss)
```

```
tensor(6434.5654, grad_fn=<MseLossBackward0>)
```

Optimizer

Instead of manually manipulating the model's weights & biases using gradients, we can use the optimizer `optim.SGD`. SGD stands for stochastic gradient descent. It is called stochastic because samples are selected in batches (often with random shuffling) instead of as a single group.

```
# Define optimizer
opt = torch.optim.SGD(model.parameters(), lr=1e-5)
```

Note that `model.parameters()` is passed as an argument to optim.SGD, so that the optimizer knows which matrices should be modified during the update step. Also, we can specify a learning rate which controls the amount by which the parameters are modified.

Train the model

We are now ready to train the model. We'll follow the exact same process to implement gradient descent:

1. Generate predictions
2. Calculate the loss
3. Compute gradients w.r.t the weights and biases
4. Adjust the weights by subtracting a small quantity proportional to the gradient
5. Reset the gradients to zero

The only change is that we'll work batches of data, instead of processing the entire training data in every iteration. Let's define a utility function `fit` which trains the model for a given number of epochs.

```
# Utility function to train the model
def fit(num_epochs, model, loss_fn, opt):

    # Repeat for given number of epochs
    for epoch in range(num_epochs):

        # Train with batches of data
        for xb,yb in train_dl:

            # 1. Generate predictions
            pred = model(xb)

            # 2. Calculate loss
            loss = loss_fn(pred, yb)

            # 3. Compute gradients
            loss.backward()

            # 4. Update parameters using gradients
            opt.step()

            # 5. Reset the gradients to zero
            opt.zero_grad()

        # Print the progress
        if (epoch+1) % 10 == 0:
            print('Epoch [{}/{}], Loss: {:.4f}'.format(epoch+1, num_epochs, loss.ite
```

Some things to note above:

- We use the data loader defined earlier to get batches of data for every iteration.
- Instead of updating parameters (weights and biases) manually, we use `opt.step` to perform the update, and `opt.zero_grad` to reset the gradients to zero.

- We've also added a log statement which prints the loss from the last batch of data for every 10th epoch, to track the progress of training. loss.item returns the actual value stored in the loss tensor.

Let's train the model for 100 epochs.

```
fit(100, model, loss_fn, opt)
```

```
Epoch [10/100], Loss: 372.2647
Epoch [20/100], Loss: 257.3275
Epoch [30/100], Loss: 64.0450
Epoch [40/100], Loss: 100.4899
Epoch [50/100], Loss: 65.0689
Epoch [60/100], Loss: 77.6008
Epoch [70/100], Loss: 31.4973
Epoch [80/100], Loss: 47.7357
Epoch [90/100], Loss: 11.1557
Epoch [100/100], Loss: 23.2684
```

Let's generate predictions using our model and verify that they're close to our targets.

```
# Generate predictions
preds = model(inputs)
preds
```

```
tensor([[ 58.3152, 71.4843],
        [ 81.4574, 100.6882],
        [118.1139, 130.6566],
        [ 28.0258, 43.5063],
        [ 96.5816, 115.5215],
        [ 58.3152, 71.4843],
        [ 81.4574, 100.6882],
        [118.1139, 130.6566],
        [ 28.0258, 43.5063],
        [ 96.5816, 115.5215],
        [ 58.3152, 71.4843],
        [ 81.4574, 100.6882],
        [118.1139, 130.6566],
        [ 28.0258, 43.5063],
        [ 96.5816, 115.5215]], grad_fn=<AddmmBackward0>)
```

```
# Compare with targets
targets
```

```
tensor([[ 56., 70.],
        [ 81., 101.],
        [119., 133.], [
        22., 37.],
        [103., 119.], [
        56., 70.], [
        81., 101.],
```

```
[119.,  133.],
[22.,    37.],
[103., 119.],
[56., 70.],
[81.,  101.],
[119.,  133.],
[22.,    37.],
[103., 119.]])
```

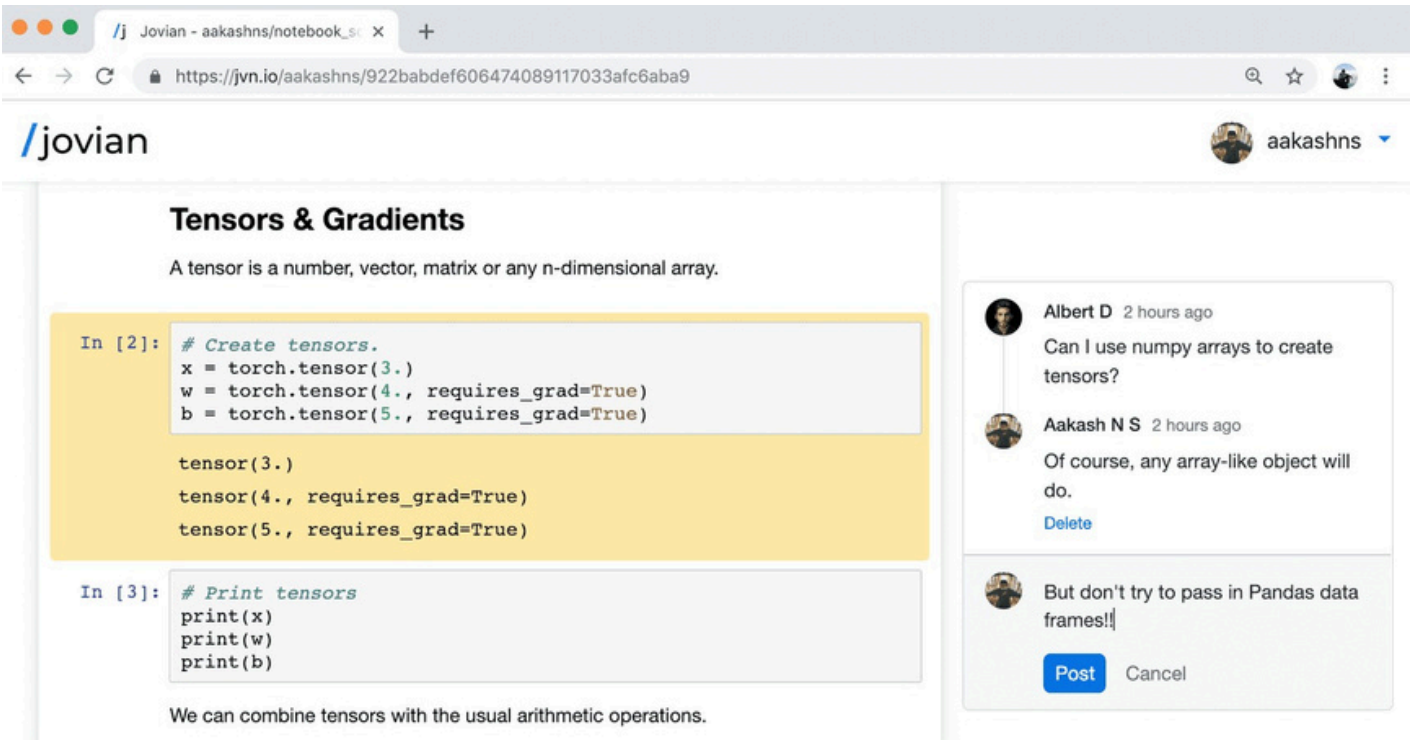
Indeed, the predictions are quite close to our targets, and now we have a fairly good model to predict crop yields for apples and oranges by looking at the average temperature, rainfall and humidity in a region.

Commit and upload the notebook

As a final step, we can save and commit our work using the jovian library.

Start coding or generate with AI.

Jovian uploads the notebook to https://jvn.io, captures the Python environment and creates a sharable link for the notebook. You can use this link to share your work and let anyone reproduce it easily with the jovian clone command. Jovian also includes a powerful commenting interface, so you (and others) can discuss & comment on specific parts of your notebook:



Further Reading

We've covered a lot of ground this this tutorial, including *linear regression* and the *gradient descent* optimization algorithm. Here are a few resources if you'd like to dig deeper into these topics:

- For a more detailed explanation of derivates and gradient descent, see these notes from a Udacity course.
- For an animated visualization of how linear regression works, see this post.
-